DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

(Artificial Intelligence & Machine Learning)

Academic Year:	2026-27	**Name of Student:**	
Semester:	V	**Student ID:**	
Class / Branch:	TE – CSE (AI&ML)	**Roll No:**	
Subject:	WMD Lab	**Date of Submission:**	

Experiment No. 07

Lab Outcome: Apply microservices, REST APIs, and deployment techniques for scalable applications.

Aim: Create RESTful APIs using Spring Boot for basic CRUD operations (users/products).

Software used: Java, Spring Boot, Postman

Theory: -

1. What is a REST API?

An Application Programming Interface (API) allows two software applications to communicate with each other. REST (Representational State Transfer) is an architectural style for designing such APIs, in which every piece of data — a student, a product, an order — is treated as a resource, uniquely identified by a URL, and manipulated using standard HTTP methods.

HTTP Method	CRUD Operation	Description
GET	Read	Retrieves one or more resources without modifying them.
POST	Create	Creates a new resource on the server.
PUT	Update (full)	Replaces an existing resource entirely.
PATCH	Update (partial)	Modifies only the specified fields of an existing resource.
DELETE	Delete	Removes an existing resource.

2. What is Spring and Spring Boot?

Spring is a widely used Java framework for building enterprise applications. Writing a plain Spring web application traditionally required a great deal of manual configuration — XML files, server setup, and dependency wiring — before a single line of business logic could run.

Spring Boot is an extension of Spring that removes most of this configuration burden. It provides auto-configuration (Spring Boot inspects the dependencies on the classpath and configures sensible defaults automatically), an embedded web server (Tomcat is bundled inside the application itself, so there is no separate server to install or deploy to), and starter dependencies (a single dependency, such as spring-boot-starter-web, pulls in every library needed for that feature).

In short — Spring Boot lets a developer focus on writing the Controller, the Entity, and the Repository, while the framework handles the server, the routing machinery, and the database wiring underneath.

3. Layered Architecture of a Spring Boot REST Application

A typical Spring Boot REST application is organised into distinct layers, each with a single responsibility:

```
HTTP Request (from Postman / browser)
            |
            ▼
     Controller Layer
(receives the request, calls the
 repository, returns a response)
            |
            ▼
     Repository Layer
(talks to the database — no SQL
     written by the developer)
            |
            ▼
       Entity / Model
(a Java class that represents
      one row of a table)
            |
            ▼
         Database
      (H2, in-memory)
```

A request enters through the Controller, which delegates data access to the Repository. The Repository operates on Entity objects, and Spring Data JPA translates those operations into database queries automatically.

4. Key Spring Boot Annotations

Annotation	Meaning
@RestController	Marks a class as a REST controller; every method's return value is automatically written directly into the HTTP response body (as JSON), instead of resolving to an HTML view.
@RequestMapping	Defines the base URL path that a controller (or a specific method) responds to.
@GetMapping / @PostMapping / @PutMapping / @PatchMapping / @DeleteMapping	Shorthand annotations that map a method to a specific HTTP method (GET, POST, PUT, PATCH, DELETE respectively).
@PathVariable	Extracts a value directly from the URL path, e.g. the 1 in /api/students/1.
@RequestBody	Converts the incoming JSON request body into a Java object automatically.
@Entity	Marks a Java class as a JPA entity — an object that maps directly onto a database table.
@Id / @GeneratedValue	Marks a field as the entity's primary key, and specifies that its value should be generated automatically by the database.
@Table	Explicitly names the database table that an entity maps to.

5. Spring Data JPA and the Repository Pattern

JPA (Jakarta Persistence API) is a specification for Object-Relational Mapping (ORM) — a technique that maps Java objects directly onto database tables, so a developer works with ordinary Java objects instead of writing raw SQL.

Spring Data JPA takes this a step further with the Repository pattern: a developer only needs to declare an interface that extends JpaRepository<EntityType, IdType>, and Spring automatically generates a working implementation at runtime, complete with methods such as save(), findAll(), findById(), deleteById(), and existsById() — without a single line of implementation code being written by hand.

This is why StudentRepository in this experiment is a completely empty interface, and yet it is fully functional: Spring Data JPA builds its behaviour dynamically from the interface's generic

type parameters.

6. The H2 In-Memory Database

H2 is a lightweight, pure-Java relational database that can run entirely in memory, requiring no separate installation, server process, or configuration. Data lives only for the lifetime of the running application and is cleared whenever the application restarts.

This makes H2 ideal for learning, prototyping, and demonstrations: a student can focus entirely on REST API design and CRUD logic, without first having to install and configure a production database such as MySQL or PostgreSQL. Spring Boot also ships an optional web-based H2 Console (enabled via spring.h2.console.enabled=true) that lets a developer inspect the in-memory tables directly in a browser.

7. HTTP Status Codes and ResponseEntity

Every HTTP response carries a status code indicating the outcome of the request. A well-designed REST API returns the correct status code for every situation, rather than always returning 200 OK:

Code	Meaning	Used When
200 OK	Success	A GET, PUT, PATCH, or DELETE request completed successfully.
201 Created	Success	A POST request successfully created a new resource.
404 Not Found	Client Error	The requested resource (e.g. a student ID) does not exist.

In Spring Boot, the ResponseEntity<T> class gives full control over both the response body and its HTTP status code in a single return value, e.g. new ResponseEntity<>(savedStudent, HttpStatus.CREATED) or ResponseEntity.notFound().build(). This is why the controller in this experiment wraps every return type in ResponseEntity, rather than returning plain Student objects.

Implementation

Experiment Title: Student Admission Portal — RESTful APIs using Spring Boot

Problem Statement

Design and build a lightweight Spring Boot RESTful Web Application for a Student Admission Portal. The system must enable admission administrators to manage student records via HTTP endpoints performing basic CRUD (Create, Read, Update, Delete) operations. The records should be stored in

an in-memory database with zero external setup, providing a simple introduction to backend web API development.

Tools Required

Tool / Software	Recommended Version	Purpose
Java Development Kit (JDK)	OpenJDK / Oracle JDK 17 or 21	Runs Java applications and the Spring Boot framework
Visual Studio Code (VS Code)	Latest version	Primary code editor
Java & Spring Extensions	Extension Pack for Java, Spring Boot Extension Pack	Syntax highlighting, autocompletion, project generation
Postman	Latest version	Testing REST API endpoints (GET, POST, PUT, DELETE)

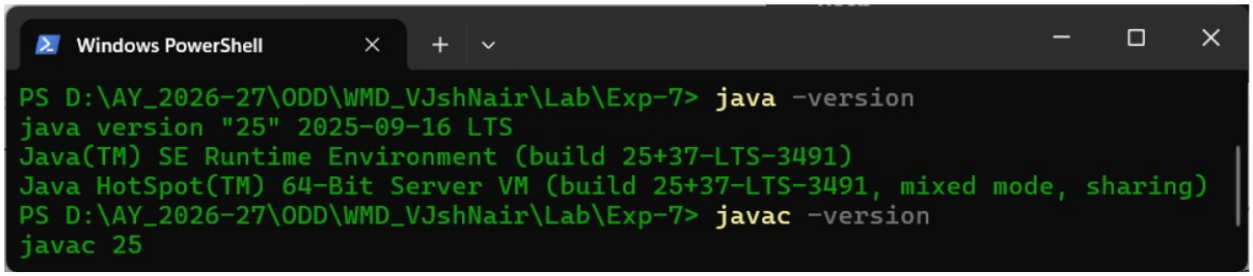
Step-by-Step Tool Download and Configuration

Step 1 — Install the Java Development Kit (JDK 17 or higher)

- Download OpenJDK (Temurin) from Adoptium, or Oracle JDK.
- Run the installer and check the box to "Set JAVA_HOME variable" during installation.

Verify the installation:

```
java -version
```



Step 2 — Install Visual Studio Code

- Download VS Code for Windows/Mac/Linux from code.visualstudio.com.
- Run the executable and complete the standard setup.

Step 3 — Install VS Code Extensions

Open VS Code, press Ctrl+Shift+X (or Cmd+Shift+X on Mac) to open the Extensions tab, and install:

- Extension Pack for Java (by Microsoft)
- Spring Boot Extension Pack (by VMware)

Step 4 — Create the Spring Boot Project in VS Code

1. Press Ctrl+Shift+P (or Cmd+Shift+P) to open the Command Palette.
2. Type "Spring Initializr: Create a Maven Project" and select it.

Choose the following settings:

- Spring Boot Version: 3.x.x (latest stable)
- Language: Java
- Group ID: com.example
- Artifact ID: admission
- Packaging: Jar
- Java Version: 17 / 21 / 25
- Dependencies: Spring Web, Spring Data JPA, and H2 Database.

Select a folder on your machine to generate the project, then open the project folder in VS Code.

Project Configuration and Code Implementation

The project follows the layered architecture introduced in the Theory section: a configuration file, an Entity that models a student, a Repository that provides database access, and a Controller that exposes the REST endpoints. Each file is explained below before its code, covering what the file does, why it matters, and how it leads into the next file in the chain.

1. Configuration File — (src/main/resources/application.properties)

Before any Java code runs, Spring Boot reads src/main/resources/application.properties on startup to decide how to configure itself — which port to listen on, and which database to connect to. This file exists so that configuration can be changed without touching any source code, which is why it is the natural starting point for the implementation.

- `server.port` sets the port the embedded Tomcat server listens on (8080 here).
- `spring.h2.console.enabled` turns on the browser-based H2 Console, useful for inspecting data while developing.
- `spring.datasource.url, driverClassName, username, password` tell Spring how to connect to the in-memory H2 database — no separate database server needs to be installed.

Why this file matters: it is what makes the same application portable across environments (development, testing, production) — only this file needs to change, never the Java code.

What comes next: once Spring Boot knows how to start its server and connect to a database, it needs to know what data actually looks like — that is the job of the Entity class.

D: > AY_2026-27 > ODD > WMD_VJshNair > Lab > Exp-7 > Student Admission Portal > admission > src > main > resources > application.properties

```
# App Name & Port
server.port=8080

# H2 Database Console Configuration
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
spring.datasource.url=jdbc:h2:mem:admissiondb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
```

2. Entity Class — (src/main/java/com/example/admission/model/Student.java)

An Entity is an ordinary Java class annotated so that Spring Data JPA can map it directly onto a database table — each field becomes a column, and each object instance becomes one row. Student.java is where the shape of a student record is defined once, and reused everywhere else in the application.

- `@Entity` tells Spring this class represents a database table.
- `@Table(name = "students")` explicitly names that table students.
- `@Id` marks id as the primary key.
- `@GeneratedValue(strategy = GenerationType.IDENTITY)` lets the database generate each new ID automatically, so the application never has to invent one itself.
- A no-argument constructor is required by JPA to create instances when loading records from the database, alongside a convenience constructor for creating new Student objects in code.
- Standard getters and setters expose the fields safely, following Java's encapsulation convention.

Why this file matters: it is the single source of truth for what a "student" is throughout the whole application — the database schema, the repository, and the controller all depend on this one class.

What comes next: defining the shape of the data is not enough on its own — the application still needs a way to actually save, fetch, update, and delete Student records. That responsibility belongs to the Repository.

D: > AY_2026-27 > ODD > WMD_VJshNair > Lab > Exp-7 > Student Admission Portal > admission > src > main > java > com > example > admission > model > Student.java

```java
package com.example.admission.model;
import jakarta.persistence.*;
@Entity
@Table(name = "students")
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    private String course;
    public Student() {}
    public Student(String name, String email, String course) {
        this.name = name;
        this.email = email;
        this.course = course;
    }
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getCourse() { return course; }
    public void setCourse(String course) { this.course = course; }
}
```

3. Repository Interface — (src/main/java/com/example/admission/repository/StudentRepository.java)

The Repository layer is responsible for talking to the database. Rather than writing SQL by hand, Spring Data JPA allows a repository to be declared as a plain interface — Spring inspects it at startup and generates a fully working implementation automatically, behind the scenes.

Extending JpaRepository<Student, Long> tells Spring two things at once: this repository manages Student entities, and each Student's primary key is of type Long. From this alone, Spring Data JPA provides save(), findAll(), findById(), deleteById(), existsById(), and more, without a single method being written.

> *Why this file matters: it eliminates almost all boilerplate data-access code. In older Java web frameworks, this same functionality could take dozens of lines of hand-written SQL and JDBC code — here it takes two lines.*

What comes next: with a way to define what data looks like (the Entity) and a way to access it (the Repository), the application is ready for the layer that actually receives HTTP requests and decides what to do with them — the Controller.

D: > AY_2026-27 > ODD > WMD_VJshNair > Lab > Exp-7 > Student Admission Portal > admission > src > main > java > com > example > admission > repository > StudentRepository.java

```java
package com.example.admission.repository;

import com.example.admission.model.Student;
import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository extends JpaRepository<Student, Long> {
}
```

4. Controller Class — (src/main/java/com/example/admission/controller/StudentController.java)

The Controller is the entry point for every HTTP request. It is responsible for interpreting the incoming request (the HTTP method, the URL, any path variables or JSON body), calling the Repository to perform the actual data operation, and returning an appropriate HTTP response — this is where the REST API defined in the Theory section becomes a working reality.

The repository is supplied to the controller through constructor injection — the StudentRepository is passed in as a constructor parameter, rather than being created inside the class with new. This is preferred over field injection because it makes the dependency explicit and mandatory, and makes the class straightforward to test in isolation by passing in a mock repository.

The class defines six endpoints, one per operation:

Method	Endpoint	Purpose
POST	/api/students	Register a new student and return it with 201 Created.
GET	/api/students	Return the full list of admitted students.
GET	/api/students/{id}	Return one student by ID, or 404 Not Found if it does not exist.
PUT	/api/students/{id}	Replace all fields of an existing student's record.
DELETE	/api/students/{id}	Remove a student's admission record.
PATCH	/api/students/{id}	Update only the fields supplied in the request body.

Each method follows the same pattern: call the repository, then use ResponseEntity to translate the outcome into the correct HTTP status code — 201 Created for a successful POST, 200 OK for a successful GET/PUT/PATCH/DELETE, and 404 Not Found when an ID does not exist in the database. The PATCH method additionally demonstrates partial updates: it accepts a generic Map<String, Object> instead of a full Student object, and only overwrites the fields that were actually supplied, leaving the rest of the record untouched.

Why this file matters: it is the only part of the application a client (Postman, a browser, or a front-end application) ever directly interacts with. Every design decision made in the Entity and Repository layers ultimately exists to support these six endpoints.

D: ❯ AY_2026-27 ❯ ODD ❯ WMD_VJshNair ❯ Lab ❯ Exp-7 ❯ Student Admission Portal ❯ admission ❯ src ❯ main ❯ java ❯ com ❯ example ❯ admission ❯ controller ❯ J StudentController.java ❯

```java
package com.example.admission.controller;

import com.example.admission.model.Student;
import com.example.admission.repository.StudentRepository;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.Map;
import java.util.List;

@RestController
@RequestMapping("/api/students")
public class StudentController {
    private final StudentRepository repository;
    // Constructor Injection
    public StudentController(StudentRepository repository) {
        this.repository = repository;
    }
    // 1. CREATE: Register a new student
    @PostMapping
    public ResponseEntity<Student> createStudent(@RequestBody Student student) {
        Student savedStudent = repository.save(student);
        return new ResponseEntity<>(savedStudent, HttpStatus.CREATED);
    }
    // 2. READ ALL: Get list of all admitted students
    @GetMapping
    public List<Student> getAllStudents() {
        return repository.findAll();
    }
    // 3. READ BY ID: Get details of a single student
    @GetMapping("/{id}")
    public ResponseEntity<Student> getStudentById(@PathVariable Long id) {
        return repository.findById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }
```

```java
    // 4. UPDATE: Modify an existing student registration
    @PutMapping("/{id}")
    public ResponseEntity<Student> updateStudent(@PathVariable Long id, @RequestBody Student updatedStudent) {
        return repository.findById(id).map(student -> {
            student.setName(updatedStudent.getName());
            student.setEmail(updatedStudent.getEmail());
            student.setCourse(updatedStudent.getCourse());
            return ResponseEntity.ok(repository.save(student));
        }).orElse(ResponseEntity.notFound().build());
    }
    // 5. DELETE: Remove student admission record
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteStudent(@PathVariable Long id) {
        if (repository.existsById(id)) {
            repository.deleteById(id);
            return ResponseEntity.ok().build();
        }
        return ResponseEntity.notFound().build();
    }
    // 6. PATCH: Partial update for student record
    @PatchMapping("/{id}")
    public ResponseEntity<Student> patchStudent(@PathVariable Long id, @RequestBody Map<String, Object> updates) {
        return repository.findById(id).map(student -> {
            if (updates.containsKey("name")) {
                student.setName((String) updates.get("name"));
            }
            if (updates.containsKey("email")) {
                student.setEmail((String) updates.get("email"));
            }
            if (updates.containsKey("course")) {
                student.setCourse((String) updates.get("course"));
            }
            return ResponseEntity.ok(repository.save(student));
        }).orElse(ResponseEntity.notFound().build());
    }
}
```

How to Run the Application in VS Code

1. Open AdmissionApplication.java (located under src/main/java/com/example/admission/).
2. Click the Run button displayed directly above the main method, or press F5.
3. The embedded Tomcat server will start on port 8080.

Expected API Testing and Output (Using Postman)

Keep the Spring Boot application running in VS Code, open the Postman desktop app, and test the REST API endpoints using the steps below. All endpoints use port 8080, matching server.port=8080 in application.properties.

1. Open a New Request Tab

Launch Postman and click the + icon near the top tabs to open a new HTTP request workspace.

2. Send a POST Request (Register Student)

- Method: select POST from the HTTP method dropdown.
- URL: http://localhost:8080/api/students

Configure the body:

- Click the Body tab beneath the URL bar.
- Select the raw option.
- Change the dropdown menu on the far right from Text to JSON.

Body data — paste the following JSON payload into the editor:

```
{
  "name": "Alex Morgan",
  "email": "alex@example.com",
  "course": "Computer Science"
}
```

Click Send.

Expected result: Status 201 Created, with a JSON response showing "id": 1.

3. Send a GET Request (View All Students)

- Click + to open a second tab in Postman.
- Method: leave as GET.
- URL: http://localhost:8080/api/students
- Click Send.

Expected result: Status 200 OK, returning a JSON array containing the registered student list.

4. Send a PUT Request (Update Record)

- Open a new tab, select PUT, and set the URL to http://localhost:8080/api/students/1.
- Select Body → raw → JSON, paste the updated student object, and click Send.

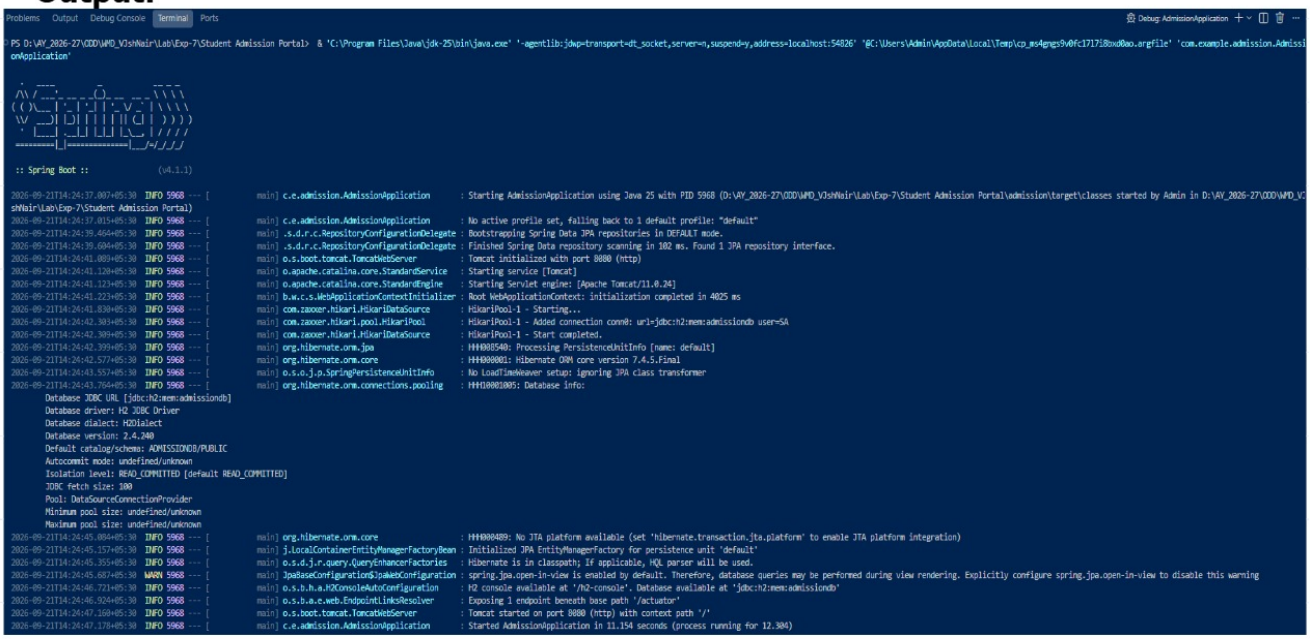
Expected result: Status 200 OK, with the updated values.

5. Send a DELETE Request (Remove Record)

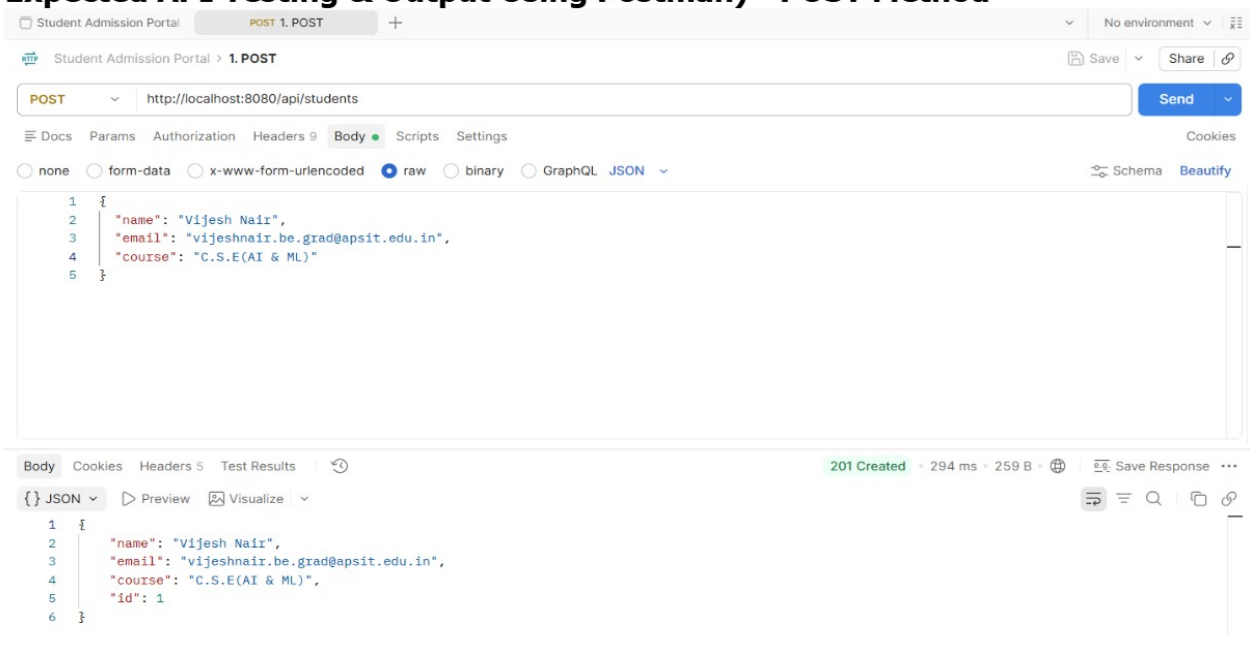
- Open a new tab, select DELETE, and set the URL to http://localhost:8080/api/students/1.
- Click Send.

Expected result: Status 200 OK.

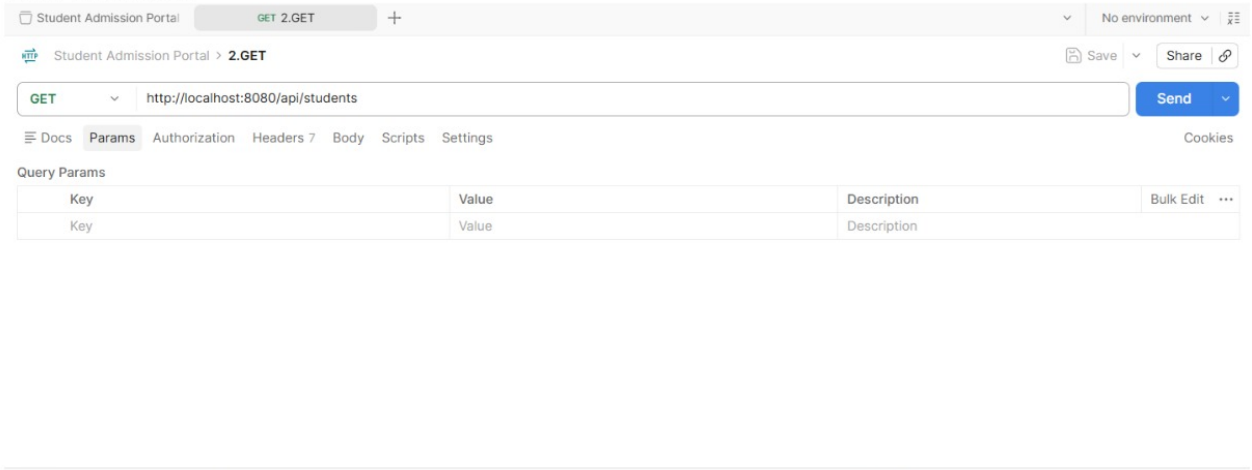
Output:

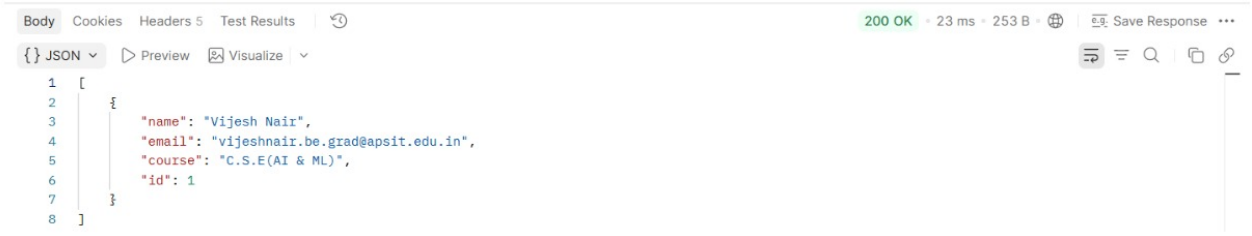


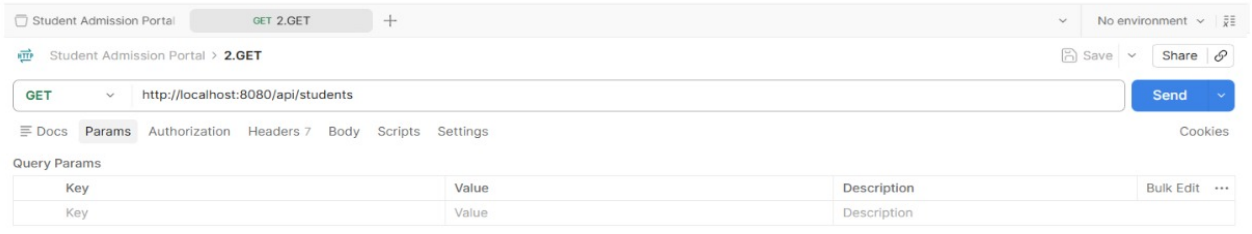
Expected API Testing & Output Using Postman) - POST Method



GET Method

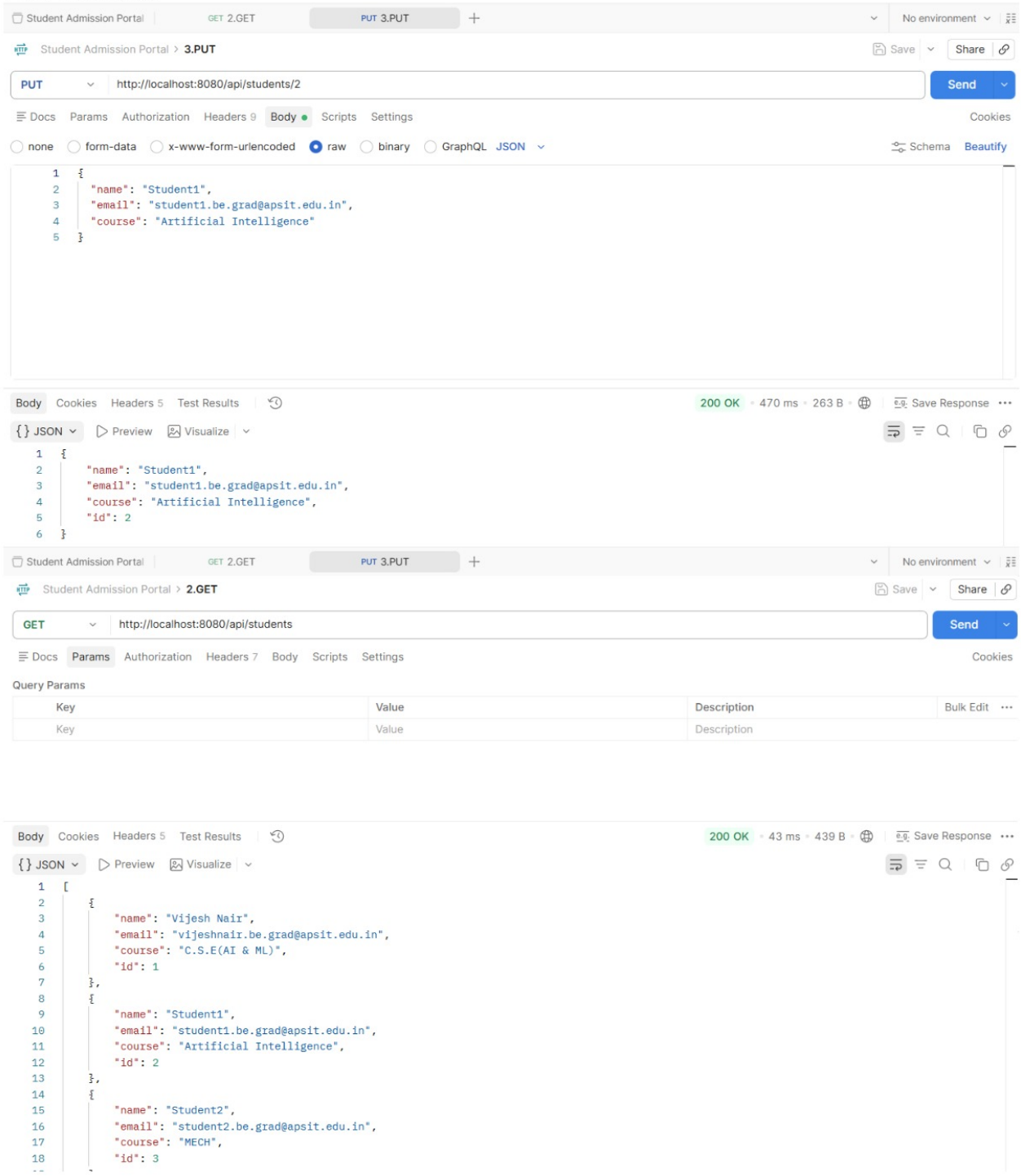

Student Admission Portal
GET 2.GET
No environment
Student Admission Portal > 2.GET
Save
Share
GET
http://localhost:8080/api/students
Send
Docs
Params
Authorization
Headers 7
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description


Body
Cookies
Headers 5
Test Results
200 OK
23 ms
253 B
Save Response
{} JSON
Preview
Visualize
[
{
"name": "Vijesh Nair",
"email": "vijeshnair.be.grad@apsit.edu.in",
"course": "C.S.E(AI & ML)",
"id": 1
}
]


Student Admission Portal
GET 2.GET
No environment
Student Admission Portal > 2.GET
Save
Share
GET
http://localhost:8080/api/students
Send
Docs
Params
Authorization
Headers 7
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description

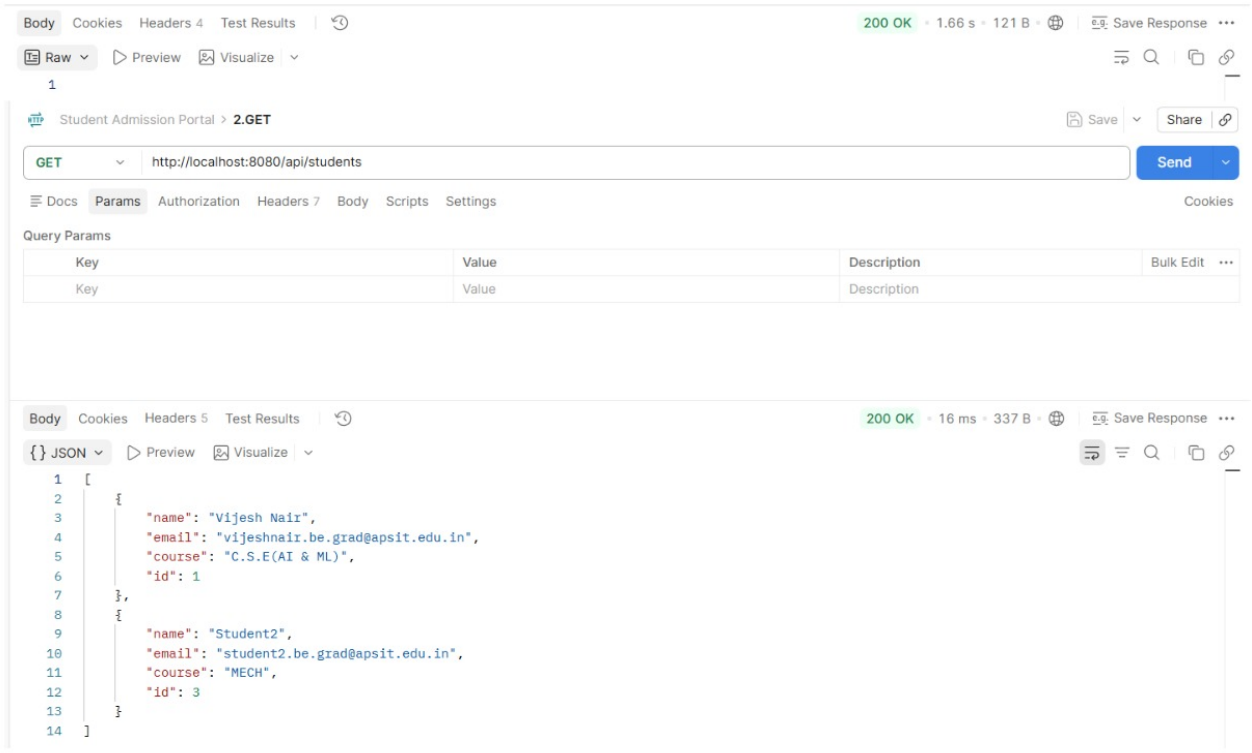

Body
Cookies
Headers 5
Test Results
200 OK
33 ms
425 B
Save Response
{} JSON
Preview
Visualize
[
{
"name": "Vijesh Nair",
"email": "vijeshnair.be.grad@apsit.edu.in",
"course": "C.S.E(AI & ML)",
"id": 1
},
{
"name": "Student1",
"email": "student1.be.grad@apsit.edu.in",
"course": "C.S.E(DS)",
"id": 2
},
{
"name": "Student2",
"email": "student2.be.grad@apsit.edu.in",
"course": "MECH",
"id": 3
}
]

PUT Method

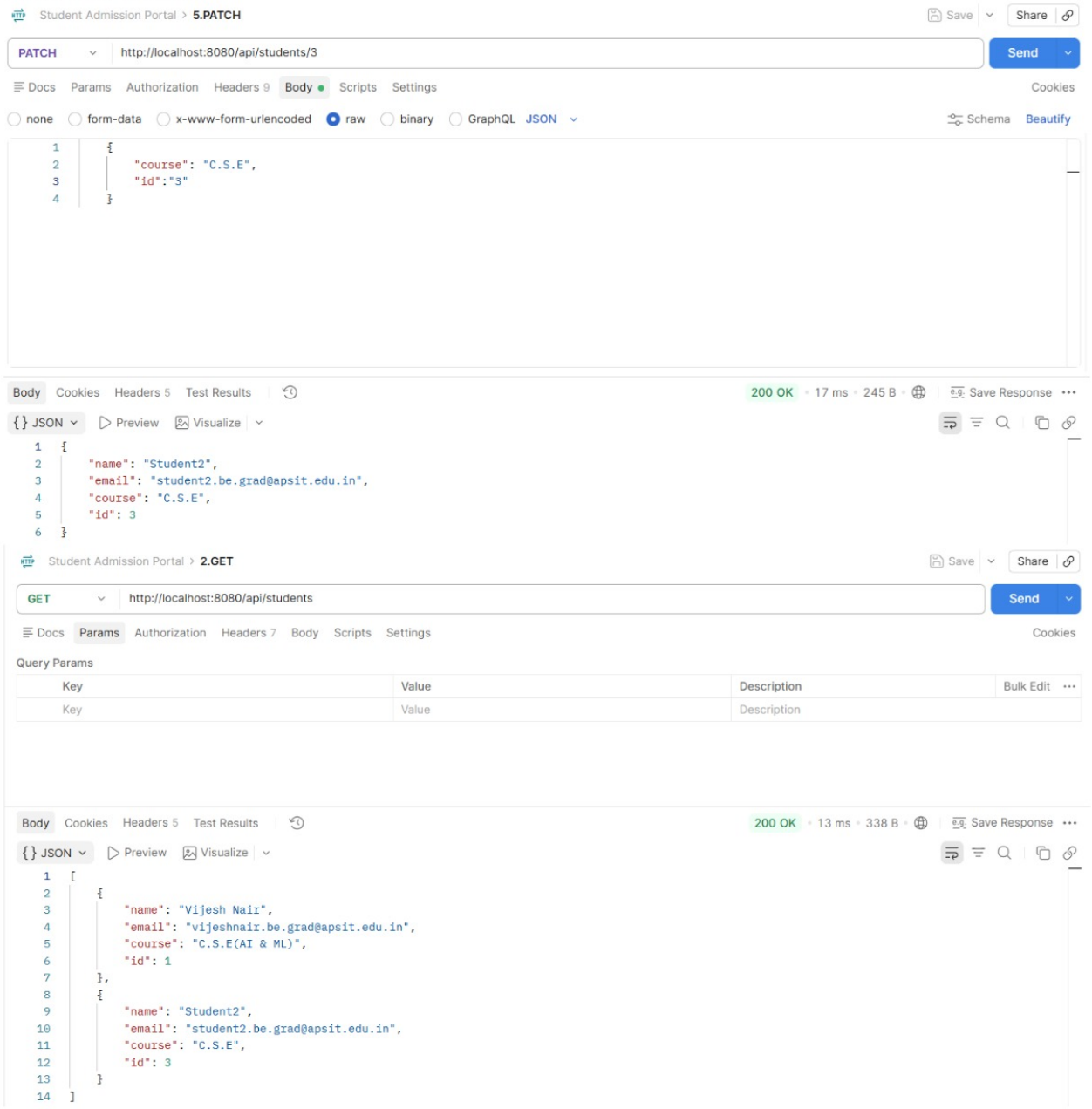

Student Admission Portal
GET 2.GET
PUT 3.PUT
No environment
Student Admission Portal > 3.PUT
Save
Share
PUT
http://localhost:8080/api/students/2
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"name": "Student1",
"email": "student1.be.grad@apsit.edu.in",
"course": "Artificial Intelligence"
}
Body Cookies Headers 5 Test Results
200 OK 470 ms 263 B
Save Response
{} JSON Preview Visualize
{
"name": "Student1",
"email": "student1.be.grad@apsit.edu.in",
"course": "Artificial Intelligence",
"id": 2
}
Student Admission Portal
GET 2.GET
PUT 3.PUT
No environment
Student Admission Portal > 2.GET
Save
Share
GET
http://localhost:8080/api/students
Send
Docs Params Authorization Headers 7 Body Scripts Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body Cookies Headers 5 Test Results
200 OK 43 ms 439 B
Save Response
{} JSON Preview Visualize
[
{
"name": "Vijesh Nair",
"email": "vijeshnair.be.grad@apsit.edu.in",
"course": "C.S.E(AI & ML)",
"id": 1
},
{
"name": "Student1",
"email": "student1.be.grad@apsit.edu.in",
"course": "Artificial Intelligence",
"id": 2
},
{
"name": "Student2",
"email": "student2.be.grad@apsit.edu.in",
"course": "MECH",
"id": 3

DEL Method


Student Admission Portal > 4.DEL
Save
Share
DELETE
http://localhost:8080/api/students/2
Send
Docs Params Authorization Headers 7 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL
This request does not have a body


Body Cookies Headers 4 Test Results
200 OK 1.66 s 121 B
Save Response
Raw Preview Visualize
1
Student Admission Portal > 2.GET
Save
Share
GET
http://localhost:8080/api/students
Send
Docs Params Authorization Headers 7 Body Scripts Settings
Cookies
Query Params
Key Value Description Bulk Edit
Key Value Description
Body Cookies Headers 5 Test Results
200 OK 16 ms 337 B
Save Response
{} JSON Preview Visualize
[
{
"name": "Vijesh Nair",
"email": "vijeshnair.be.grad@apsit.edu.in",
"course": "C.S.E(AI & ML)",
"id": 1
},
{
"name": "Student2",
"email": "student2.be.grad@apsit.edu.in",
"course": "MECH",
"id": 3
}
]

PATCH Method



Conclusion: -

Thus, we have demonstrated and implemented RESTful APIs using Spring Boot for basic CRUD operations (users/products).